Research Answers

1. HTTP is a stateless protocol, which means that each request and response is treated independently and no information about previous interactions is automatically retained. To maintain user state, particularly for authentication, web applications use sessions, cookies, and authentication tokens.

   When a user logs in, the server creates a session and generates a unique session ID. This session ID is typically stored in a cookie on the user's browser and is included with every subsequent request. The server uses the session ID to retrieve the user's session data, which may be stored in memory or a database.

   In Django, session management is handled automatically through middleware that processes session cookies. Django's authentication system links the authenticated user to the session, enabling the framework to recognise the user across multiple requests. Another common approach is token-based authentication, such as JSON Web Tokens (JWTs), where a signed token containing user information is stored on the client and sent with each request. These techniques allow web applications to securely maintain user state while working within the stateless nature of HTTP.

2. To migrate a Django application to a server-based relational database such as MariaDB, the database software must first be installed and configured. A database and user account are then created, with the necessary permissions assigned to allow Django to access and manage the data.

   Next, the DATABASES setting in Django’s settings.py file is updated to connect to MariaDB. This typically requires installing a database connector such as MySQL client and providing the connection details, including the database name, username, password, host, and port.

   After the database connection has been configured, Django’s migration tools are used to create and apply the database schema. The python manage.py make migrations command generates migration files based on changes made to the application's models, while python manage.py migrate applies those changes to the MariaDB database.

   Django records all applied migrations in a dedicated migration history table, allowing it to track which changes have already been implemented. This ensures that migrations are executed only once and in the correct sequence. By using Django’s migration framework, developers can manage database schema changes efficiently while maintaining consistency between the application models and the underlying relational database.